Assignment 2 – Theoretical Questions

Q1.1 – No, As define only "gives a name" to an expression that is composed of primitives. Assuming we defined a variable x as an expression in L1, we could replace x everywhere it is used with its actual value and it will be an equivalent L11 program.

Q1.2 – No, same as in Q1.1, only expanded to functions. Assume we define a function f as a lambda function, we could replace f everywhere it is used with the lambda expression and it will be an equivalent L2 program. In the unique case of recursive functions, we will add a parameter which will be the "step" of the recursion, and the main function will run until it reaches the stop condition. The "define" keyword is used mainly for convenience and order.

Q1.3 – No, here we can use the concept of "Currying" – every lambda expression with x parameters can be transitioned into x lambda expressions with 1 parameter. As for the body, we can "nest" all the expressions one inside the other using lambdas, each one gets the previous expression as the argument and returns the next one. That way, the program is a legitimate L22 program and is equivalent to the L2 program.

Q1.4 – Yes, there are programs that cannot be transformed, for example:

```
((lambda (f x) (f x)) (lambda (y) (* y y)) 3)
```

Q1.5 – Special forms are required to control the evaluation order. Primitive operators evaluate all their arguments before applying the operation (applicative order). If structures like "if" were primitive operators, both the "then" and "else" expressions would be evaluated before the condition is checked. This would cause runtime errors or infinite loops in recursive calls, making the language not Turing-complete.

For example, (if (= x 0) 1 (/ 10 x)).

Q1.6 – No, it is not required, as in functional programming there are no side-effects and only the last expression in the function is returned, while the previous expressions do not affect the program. In other paradigms such as imperative, it is required, for example printing to the user or changing values of variables. In L3, it is possible to have multiple expressions, but as it is a functional-programming language, it is not necessary.

Q1.7 – A lexical address of a variable indicates: 1. How many "levels" do you need to go up to get to the scope in which the variable was declared and 2. Its order of declaration in that scope. If a variable wasn't declared, it is considered free. For example, if we have this program:

```
(lambda (y x)
        ((lambda (x) (+ x y))
                (+ x z)))
```

The lexical addresses are: [x: 0 0], [y: 1 0] in the internal lambda and [x: 0 1] [z: free] in the external.

Q1.8 – One advantage for each:

a. PrimOp – Easier and less complicated calculation, no need for frame creation or variable substitution.
b. Closure – If we would make every operator a closure, there is no need to check if the operator is a primitive or a closure, thus making the interpreter work less and more efficient.

Q1.9 –

a. Case 1 – When there is no need to calculate all the expressions.
   For example – ((lambda (x y z) (if x y z) #t 3 (sqrt 2)))
   Case 2 – When one of the expressions leads to an error.
   For example – ((lambda (x y z) (if x y z) #t 3 (/ 30 0)))
   Case 3 – When an expression is recursive, applicative can cause an infinite loop, as opposed to normal in which the program stops correctly.
   For example - (define loop (lambda () (loop)))
   (define f (lambda (x) 5))
   (f (loop))
b. Case 1 – When the same expression is used multiple times.
   For example - ((lambda (x) (+ x x)) (* 2 3))

Q1.10 –

a. The role of the function valueToLitExp is to convert the values of the arguments that were sent to the closure back to literal expressions, so that we could substitute the values later correctly, since the substitute function only knows how to work with literal expressions (CExp).
b. The function is not needed in normal evaluation strategy because the values of the arguments are not calculated yet when using the substitute method, so they are still literal expressions.
c. The function is not needed in the environment-model interpreter because the model does not use the substitute function, but reads the values from the environment.

Q1.11 –

a. Renaming is not required in the environment model because each procedure has access to all the environments that exited at the time of its creation, and will choose for each parameter the latest value. That way, there is no confusion with variable names.
b. No, the renaming function is not needed in that case, as the use of the function is for cases in which a free variable is mistaken for a bound variable and is assigned the wrong value. Since there are no free variables, there is no need for name changing.

Q2.d –

```
(define pi 3.14)
(define square (lambda (x) (*x x)))
(define circle
        (lambda (x y radius)
                (lambda (msg)
                        (if (eq? msg 'area)
                                (* (square radius) pi)
                                (if (eq? msg 'perimeter)
                                        (* 2 pi radius)
                                        'error)))))
(define c (circle 0 0 3))
(c 'area)
```

The expressions:

```
(circle 0 0 3)
circle
0
0
3
(lambda (msg)
        (if (eq? msg 'area)
                (* (square 3) pi)
                (if (eq? msg 'perimeter)
                        (* 2 pi 3)
                        'error)))))
(c 'area)
c
'area
(if (eq? 'area 'area)
        (* (square 3) pi)
```

```
        (if (eq? 'area 'perimeter)
                (* 2 pi 3)
                'error))
```

(eq? 'area 'area)

eq?

'area

'area

(* (square 3) pi)

*

(square 3)

Square

3

(* 3 3)

*

3

3

pi

Environment Diagram: